

Introduction to Machine Learning

Lecturer: Abdan Hafidz

Topic: Fundamentals of Computational Learning Theory

Level: Undergraduate / Olympiad Preparation (IOAI)

1 Computation and the Paradigm Shift

1.1 The Deterministic Limitation

The foundation of classical computer science rests upon the concept of **Deterministic Algorithms**. In this traditional paradigm, a computer functions as a rigid logical processor that executes a sequence of explicit, pre-defined instructions. The logic of the transformation is strictly "hard-coded" by the programmer, who acts as the primary intelligence in the loop. The programmer must possess a complete, analytical understanding of the problem domain to manually derive the function f that maps an input space $\mathcal{X}$ to an output space $\mathcal{Y}$.

Formally, a classical algorithm A is defined as a mapping where the relationship is fixed, known, and yields a guaranteed result with zero uncertainty:

$$f : \mathcal{X} \rightarrow \mathcal{Y} \quad \text{such that} \quad y = f(x)$$

Consider the arithmetic operation of summing two integers ($1 + 1 = 2$). This is an analytic and deterministic process; the rules are universal, and the outcome is invariant. However, this rigidity becomes the Achilles' heel of classical computing when applied to complex, real-world systems where the rules are not immediately apparent or are too complex to define manually.

1.2 The Stochastic Reality

Real-world phenomena—such as plant growth dynamics, meteorological patterns, or financial market fluctuations—are rarely deterministic. Instead, they are **Stochastic**. This means they possess inherent randomness or "noise" caused by unobserved variables ϵ (epsilon).

In such a stochastic environment, we cannot define a perfect, deterministic function $y = f(x)$. A single input x does not guarantee a single fixed output y because of these hidden factors. Instead, we must model the system using a conditional probability distribution:

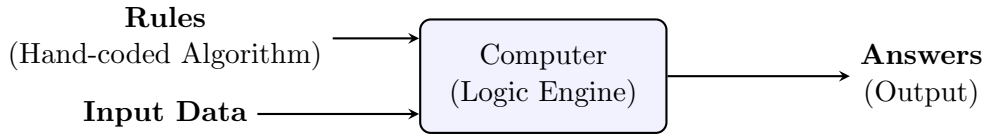
$$P(Y|X) \quad \text{or} \quad y = f(x) + \epsilon$$

Machine Learning (ML) represents the computational paradigm designed to approximate these stochastic relationships. It inverts the traditional interaction loop: instead of providing the computer with the rules (f), we provide the system with empirical data (examples of x and y) and instruct it to *learn* the rules by finding patterns within the noise.

1.3 Visualizing the Paradigm Shift

To crystallize the difference between these two approaches, we can visualize the flow of information. The fundamental shift lies in the source of the "Logic" or "Rules."

Classical Programming



Machine Learning

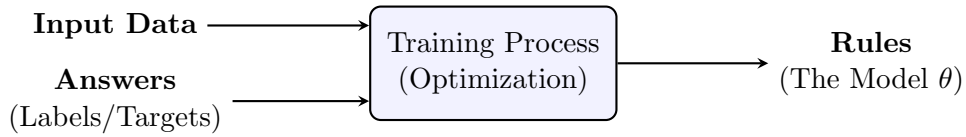


Figure 1: The Paradigm Inversion: Classical programming requires humans to input the rules to process data. Machine Learning uses data and the answers to discover the rules (the Model) automatically.

In the **Classical** paradigm, the programmer provides the intelligence. In the **Machine Learning** paradigm, the data provides the intelligence, and the computer’s role is to extract it.

2 Mathematical Methodologies in Learning

The transition from classical computing to Artificial Intelligence involves a fundamental shift in mathematical methodology. We can categorize problem-solving approaches into three distinct classes, each with varying applicability to AI.

2.1 Analytical Methods

An **Analytical Method** attempts to solve a problem by deducing an exact mathematical formula or closed-form solution. This approach relies on algebraic manipulation to derive a perfect answer directly. For example, given two points on a Cartesian plane, one can analytically derive the exact equation of the line $y = mx + c$ by calculating the slope:

$$m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$

While mathematically elegant, analytical methods are severely limited in the context of AI. They require the relationship between variables to be simple, perfectly defined, and noise-free—conditions almost never met in real-world data.

2.2 Numerical Methods (Iterative Approximation)

In contrast, Machine Learning predominantly relies on **Numerical Methods**. Rather than attempting to “jump” straight to the exact answer using a formula, the computer employs a strategy of “iterative approximation”—crawling towards the solution step-by-step.

To perform this approximation, the machine learning model needs internal settings that it can adjust. **What is θ ?**

Think of a machine learning model as a "black box" equipped with a set of control knobs. By turning these knobs, you change how the machine processes input data.

- If the machine predicts poorly, it means the knobs are in the wrong positions.
- To fix it, we tweak the knobs slightly to reduce the error.

In mathematical notation, it is tedious to list every single knob individually (e.g., knob 1, knob 2, ... knob 1000). Instead, we group **all** these adjustable parameters into a single universal symbol: θ (Theta).

Definition: θ represents the entire set of learnable parameters within a model. When we "initialize the model," we are simply setting θ to random starting values. The learning process is the mathematical act of iteratively updating θ :

$$\theta_{new} \leftarrow \theta_{old} + \text{small adjustment}$$

This process, often referred to as "Optimization," is simply the act of twisting these knobs (θ) until the model's output matches the real-world data.

2.3 Stochastic Methods

To handle the inherent randomness of real-world data, modern learning algorithms incorporate **Stochastic Methods**. These methods do not assume a perfect correlation between input and output. Instead, they estimate the expectation of the target variable:

$$\hat{y} = \mathbb{E}[Y|X]$$

This allows the model to make predictions that represent the *most likely* outcome rather than a guaranteed certainty, which is crucial for handling outliers and noise within the dataset.

3 Data: The Core Representation

3.1 Formal Definition and Dimensions

Data serves as the primary input for any learning algorithm. Formally, a dataset $\mathcal{D}$ is defined as a set of N tuples, drawn from the product space of inputs $\mathcal{X}$ and labels $\mathcal{Y}$:

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N \subset \mathbb{R}^D \times \mathbb{R}$$

To process this information computationally, data is organized into matrices. Let N represent the number of samples (rows) and D represent the number of features (columns). The dataset is thus represented as a Feature Matrix $\mathbf{X}$ and a Label Vector $\mathbf{Y}$.

Visual Abstract of the Feature Matrix ($\mathbf{X}$):

The matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ represents the entire input space:

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,D} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N,1} & x_{N,2} & \cdots & x_{N,D} \end{bmatrix}$$

3.2 Data Abstraction and Vectorization

Computers cannot natively understand abstract concepts like images or language; they can only process numerical tensors. Therefore, we define a feature extraction function ϕ that maps raw data to a vector space:

$$\phi : \text{RawData} \rightarrow \mathbb{R}^D$$

The table below illustrates this transformation across different domains:

Data Type	Real-World Entity	Computational Abstraction	Dims	Example Feature
Tabular	Spreadsheet Row	Vector of discrete values $[x_1, \dots, x_D]$	$1 \times D$	x_1 : Age, x_2 : Salary
Image	Photograph	Flattened matrix of pixel intensities	$1 \times (HWC)$	x_i : RGB Value (0-255)
Text	Sentence	Vector Embeddings	$1 \times V$	x_i : Frequency of "Fox"

3.3 Visual Illustration of Dataset Structure

To visualize how this mathematical abstraction translates into a structured dataset used in training, consider the tabular representation below. Each row corresponds to a single observation (sample), and each column corresponds to a feature or the target label.

Table 1: Structure of a Generic Dataset $\mathcal{D}$ (N Samples, D Features)

Sample ID (i)	Feature 1 (x_1)	Feature 2 (x_2)	...	Feature D (x_D)	Label (y)
1	$x_{1,1}$	$x_{1,2}$	...	$x_{1,D}$	$y^{(1)}$
2	$x_{2,1}$	$x_{2,2}$	...	$x_{2,D}$	$y^{(2)}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
N	$x_{N,1}$	$x_{N,2}$	...	$x_{N,D}$	$y^{(N)}$

3.4 Concrete Example: From Table to Matrix

To ground these abstract definitions in a concrete scenario, let us revisit the simple numerical example from the lecture. Consider a dataset generated by the underlying function $y = 2x + 1$ (where x is a single feature).

Table 2: Concrete Example: Simple Linear Dataset

Sample ID	Feature (x)	True Label (y)
1	1	3
2	2	5
3	3	7

In the context of the matrix definitions provided in Section 3.1, this tabular data is transformed into the following algebraic structures:

$$\mathbf{X} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \in \mathbb{R}^{3 \times 1}, \quad \mathbf{Y} = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} \in \mathbb{R}^{3 \times 1}$$

This specific representation is critical because the machine learning model (Section 4) will not "see" the table; it will exclusively process these matrices $\mathbf{X}$ and $\mathbf{Y}$. The goal of the learning process (Section 5) is to mathematically derive the parameters $\mathbf{w} \approx [2]$ and $b \approx 1$ solely by analyzing these two matrices.

4 From Explicit Algorithms to Learning Models

4.1 The Linear Model Hypothesis

The fundamental building block of machine learning is the **Linear Model**. We hypothesize that the output y is a weighted linear combination of the input features $\mathbf{x}$. The equation governing this relationship for a single sample is:

$$y = \mathbf{w}^T \mathbf{x} + b$$

In this equation, $\mathbf{w} \in \mathbb{R}^D$ represents the **Weights**, a vector of trainable parameters that determine the importance or "slope" of each feature. The parameter $b \in \mathbb{R}$ represents the **Bias**, which serves as an intercept.

4.2 Geometric Visualization

To visualize the relationship between the input x and the output y , we can plot the data points on a Cartesian plane. The "Learning" process is geometrically equivalent to finding the line that passes closest to these points.

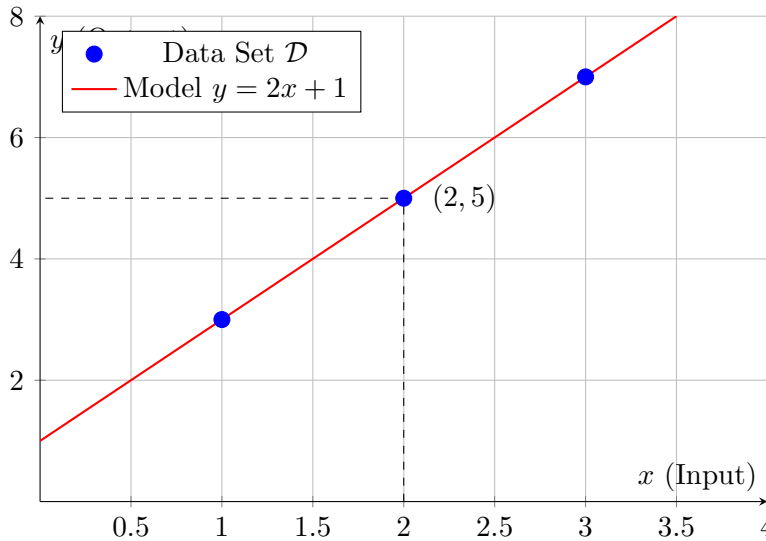


Figure 2: Geometric interpretation of the linear hypothesis fitting the dataset. The red line represents the best-fit model where $\mathbf{w} = 2$ and $b = 1$.

The equation $y = 2x + 1$ visualized above provides a clear analytical meaning to the parameters:

- **Slope ($\mathbf{w} = 2$):** The rate of change. Analytically, $m = \frac{\Delta y}{\Delta x} = \frac{5-3}{2-1} = 2$. This confirms that for every unit increase in x , y increases by 2.
- **Intercept ($b = 1$):** The point where the line crosses the y -axis (when $x = 0$).

4.3 The Challenge of Real-World Data: Stochasticity and Hypotheses

While the previous example showed perfect alignment, real-world data is inherently **continuous** and often appears **patternless** or noisy due to stochastic factors. In such scenarios, data points do not lie perfectly on a single line but form a scattered distribution.

This stochasticity creates a fundamental difficulty: there is no single line that passes through every point. Instead, we are faced with an infinite number of possible lines, each representing a different potential model. In Machine Learning terms, each potential line is called a **Hypothesis**, denoted as h .

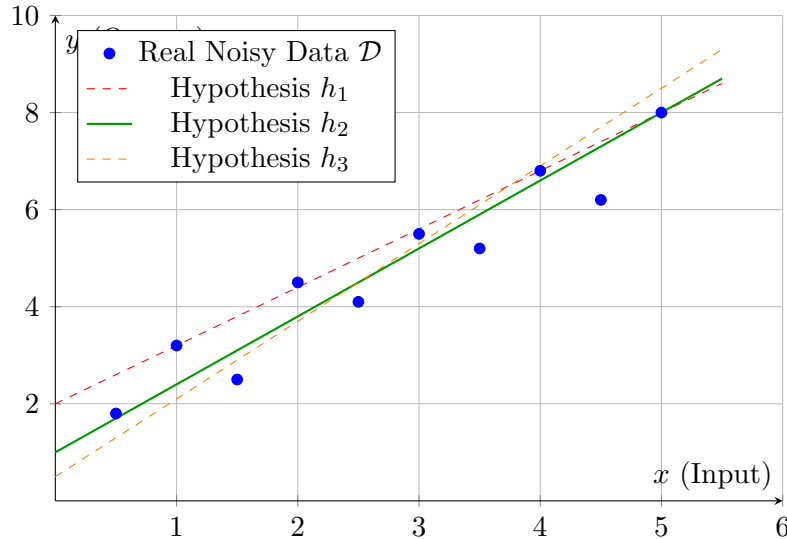


Figure 3: Visualization of Real-World Stochastic Data. Unlike the perfect example, the data points are scattered. We can draw multiple lines (h_1, h_2, h_3) that roughly follow the trend, but none fit perfectly. The goal of learning is to identify which specific hypothesis is the "best" approximation of the underlying pattern.

In the figure above, h_1 , h_2 , and h_3 are all valid candidates in the **Hypothesis Space**. They represent different configurations of our parameters $\mathbf{w}$ and b . The existence of this variability necessitates a formal, mathematical method to evaluate "goodness of fit," which leads us to the concepts of Error and Loss functions discussed in Section 5.

4.4 Matrix Formulation for Batch Processing

In a real computational setting, we process the entire dataset (or batch) simultaneously using Linear Algebra. The prediction vector $\hat{\mathbf{Y}}$ is calculated via a matrix-vector multiplication.

Given Input Matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$, Weight Vector $\mathbf{w} \in \mathbb{R}^{D \times 1}$, and Bias b :

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{w} + b$$

Expanded Abstract Representation:

$$\underbrace{\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}}_{N \times 1} = \underbrace{\begin{bmatrix} x_{1,1} & \cdots & x_{1,D} \\ x_{2,1} & \cdots & x_{2,D} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,D} \end{bmatrix}}_{N \times D} \times \underbrace{\begin{bmatrix} w_1 \\ \vdots \\ w_D \end{bmatrix}}_{D \times 1} + \underbrace{b}_{\text{Bias}}$$

4.5 Generalization of Parameters: The Transition to θ

In introductory algebra, we separate the slope (m or $\mathbf{w}$) and the intercept (c or b). However, in advanced machine learning, managing separate variables for weights and biases becomes mathematically unwieldy, especially when calculating derivatives for optimization.

To streamline our notation and calculations, we group all learnable components into a single **Unified Parameter Set**, which we identified in Section 2 as θ (Theta).

Key Concept: Unified Notation

Think of θ as a container that holds all the "knobs" of the machine.

$$\theta = \{\mathbf{w}, b\}$$

By treating the weights and bias as a single mathematical entity θ , we can express the hypothesis function $h(\mathbf{x})$ more elegantly:

$$h_{\theta}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

From this point forward, when we refer to "optimizing the model," we are referring to finding the optimal θ . This abstraction allows us to express the Loss Function simply as $J(\theta)$ rather than $J(\mathbf{w}, b)$, enabling us to write the gradient update as a clean, single-vector operation.

5 Learning as Optimization

5.1 Defining Error, Loss, and Cost

To automate the learning process, we must strictly quantify "how wrong" the model's predictions are. This quantification occurs at three hierarchical levels:

1. Error (e): The residual difference between the prediction and the actual label for a *single* data point:

$$e^{(i)} = h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}$$

2. Loss Function ($J(\theta)$): This metric aggregates the errors over the entire dataset. The standard metric for regression is the **Mean Squared Error (MSE)**. We square the errors for a specific reason: to penalize large errors more heavily and to ensure that positive and negative errors do not cancel each other out (e.g., an error of +5 and -5 should not average to 0).

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

5.2 The Mathematical Cycle of Learning

The learning process is formally defined as an iterative algorithm. For each iteration t , the system performs three distinct mathematical operations in sequence:

1. Forward Propagation: $\hat{\mathbf{y}}^{(t)} = h_{\theta^{(t)}}(\mathbf{X})$ (Make a guess)
2. Cost Evaluation: $J(\theta^{(t)}) = \frac{1}{N} \|\hat{\mathbf{y}}^{(t)} - \mathbf{y}\|^2$ (Score the guess)
3. Backward Propagation: $\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} J(\theta^{(t)})$ (Improve the guess)

Where:

- α (Alpha) represents the **Learning Rate** or step size. It controls how fast we move.
- ∇_{θ} (Nabla) represents the **Gradient** vector. This points in the direction of steepest ascent. By subtracting it, we move downhill towards lower error.

The ultimate goal of training is to minimize this Cost:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

6 Generalization: Validation and Overfitting

6.1 The Deception of Training Performance

A critical concept in machine learning is that performance on training data is deceptive. A model can achieve near-zero error on the training set simply by "memorizing" the answers. This is mathematically equivalent to a high-degree polynomial passing through every data point but oscillating wildly in between.

6.2 Data Splitting: Training vs. Testing

To measure true learning—**Generalization**—we must rigorously split the dataset $\mathcal{D}$ into two disjoint subsets. The key is that the "Test" data must act as a proxy for the future—data the model has never seen before.

$$\mathcal{D} = \mathcal{D}_{train} \cup \mathcal{D}_{val} \quad \text{where} \quad \mathcal{D}_{train} \cap \mathcal{D}_{val} = \emptyset$$

- **Training Set** ($\mathcal{D}_{train}$): Used to optimize θ . The model "learns" from this.
- **Validation Set** ($\mathcal{D}_{val}$): Used strictly for evaluation. The model is "tested" on this.

6.3 Validation Loss and Overfitting

Validation Loss is the error calculated on the unseen Validation Set. Formally:

$$J_{val}(\theta) = \frac{1}{|\mathcal{D}_{val}|} \sum_{(\mathbf{x}, y) \in \mathcal{D}_{val}} (h_{\theta}(\mathbf{x}) - y)^2$$

We detect **Overfitting** by comparing the training loss J_{train} and validation loss J_{val} . Overfitting occurs when:

$$J_{train}(\theta) \approx 0 \quad \text{and} \quad J_{val}(\theta) \gg J_{train}(\theta)$$

This inequality implies that the model has learned the noise in the training data rather than the underlying signal, failing to generalize to new $\mathbf{x}$ inputs.